

Otimização e Controle de Concorrência na Estimativa Estocástica de π utilizando OpenMP

Resumo

Este relatório apresenta a implementação e otimização do cálculo de π através do método de Monte Carlo (estimativa estocástica) em linguagem C, utilizando a API OpenMP. O objetivo do estudo é analisar as anomalias introduzidas pelo acesso não sincronizado a regiões críticas (condição de corrida), o impacto do uso de `#pragma omp critical` no desempenho e, por fim, a correta reestruturação do código mediante as cláusulas de escopo de dados (`private`, `firstprivate`, `lastprivate` e `shared`). Também é discutida a importância da diretiva `default(none)` em programas complexos paralelos.

1. Introdução

A estimativa do número π pelo método de Monte Carlo baseia-se no sorteio aleatório de pontos no primeiro quadrante de um plano cartesiano com raio unitário. A proporção de pontos que caem dentro do círculo (onde $x^2 + y^2 \leq 1$) em relação ao total de pontos aproxima-se da área do setor circular ($\pi/4$).

Devido à independência entre cada iteração do sorteio, o algoritmo é altamente paralelizável. No entanto, o somatório (acumulação) requer acesso concorrente a uma variável compartilhada, exigindo mecanismos adequados de sincronização para evitar *race conditions* (condições de corrida) e manter a eficiência e a correção.

2. Metodologia e Abordagens

O código foi escrito em C e compilado com o `gcc` (flag `-fopenmp` para ativação das diretivas). Para evidenciar os efeitos da concorrência, o algoritmo foi subdividido em três abordagens:

- Abordagem Ingênua (Condição de Corrida):** Um simples `#pragma omp parallel for` é adicionado sobre o laço de cálculo, permitindo acesso não sincronizado à variável `count`.
- Abordagem com Sincronização Estrita (`omp critical`):** A variável compartilhada `count` é sincronizada a cada iteração, garantindo resultados corretos à custa de um gargalo drástico na performance.
- Abordagem Reestruturada com Cláusulas de Escopo:** O cálculo é particionado localmente em cada thread. Usam-se as cláusulas `shared`, `private`, `firstprivate` e `lastprivate` explicitamente controladas via `default(none)`.

A geração de números pseudoaleatórios é adaptada utilizando a função reentrante `rand_r()` em conjunto com a identificação local da thread para definir as "sementes" matemáticas com segurança perante o multithreading.

3. Resultados e Discussão

Os testes foram executados com $N = 10.000.000$ interações. Abaixo são apresentados e discutidos os três resultados obtidos:

3.1. Condição de Corrida

O uso puramente ingênuo do `#pragma omp parallel for` gerou o seguinte cenário:

- **n estimado:** 1.382396
- **Tempo gasto:** 0.129760 segundos

Explicação do Erro: O valor calculado dista acentuadamente do n real (~ 3.14159). Como várias threads tentam acessar e modificar a mesma posição de memória `count` simultaneamente (a instrução `count++` engloba leitura, incremento e gravação), ocorrem sobrescritas acidentais de valores perdendo muitas contagens legítimas. Embora a execução tenha sido veloz, o resultado final é intrinsecamente incorreto devido à condição de corrida (*data race*).

3.2. Cláusula Critical

Buscando corrigir a condição de corrida sem alteração de escopo, inseriu-se um `#pragma omp critical` englobando a operação `count++`:

- **n estimado:** 3.141608 (Correto)
- **Tempo gasto:** 2.817176 segundos

Análise do Gargalo: O resultado retornado é agora correto. Contudo, o bloco crítico obriga que o acesso de cada thread seja estritamente sequencial dentro do laço paralelo. Considerando a altíssima frequência em que os pontos atendem à condição do círculo (aproximadamente 78% das interações), a sobrecarga de bloqueio e liberação do semáforo entre as threads causou um forte gargalo (*bottleneck*). Ironicamente, o tempo de execução foi drasticamente penalizado, tornando a "paralelização" ineficiente (muito mais demorada do que se não houvesse paralelismo algum).

3.3. Reestruturação com Cláusulas de Escopo (`private`, `firstprivate`, `lastprivate` e `shared`)

A terceira implementação aborda a eficiência delegando os contadores localmente e realizando a soma ao final, utilizando diferentes cláusulas de dados para ilustrar seus efeitos na arquitetura OpenMP:

- **n estimado:** 3.141377 (Correto)
- **Tempo gasto:** 0.025093 segundos

O desempenho saltou de quase 3 segundos na versão *critical* para rápidos ~ 25 milissegundos. Isto porque o estado local do iterador e as próprias interações ocorrem independentemente e com máxima eficiência. O acesso protegido (*critical*) ocorre apenas uma única vez por thread, ao finalizar a região paralela.

Efeito de cada cláusula introduzida no teste:

- **`private(x, y)`:** Garante que cada thread tenha a sua própria cópia não inicializada das variáveis `x` e `y` no for, prevenindo o compartilhamento de valores momentâneos entre iterações paralelas de threads diferentes.
- **`firstprivate(first_priv_var)`:** Inicializa uma nova variável privada com o exato valor que essa mesma variável continha imediatamente antes do escopo paralelo. No nosso teste, a variável reteve o valor prévio de inicialização (100) garantindo leitura coerente no processamento interno da thread.
- **`lastprivate(last_priv_var)`:** Opera similarmente à cláusula *private*, porém encarrega a thread incumbida de rodar a "sequencialmente última iteração lógica" do laço de for ($i == N - 1$) a atualizar a

variável externa correspondente. No nosso teste a variável exportou o ID da thread da última passagem do laço.

- **shared(count, shared_var)**: A cláusula *shared* especifica acesso simultâneo na memória global original, usada para guardar informações transversais e consolidadas a todos os recursos de processamento (neste caso, atualizamos os resultados numéricos **count** e **shared_var** via bloco critical para manter coerência sem perda de dados).

3.4 O Papel de **default(none)**

O uso explícito de **default(none)** força o programador a declarar manualmente e de forma mandatória a restrição de dados (seja *shared*, *private*, *firstprivate* etc.) para todas as variáveis usadas no interior do bloco paralelo, impedindo suposições equivocadas ou implícitas por parte do compilador. Essa prática ajuda muito a tornar o escopo explícito em sistemas computacionais complexos, minimizando severamente a introdução acidental de falhas lógicas e *race conditions* silenciosas decorrentes do vazamento de escopo.

4. Conclusão

A paralelização de algoritmos em C com OpenMP exige análise profunda das interdependências das variáveis de memória do bloco. Evidencia-se que o uso indiscriminado da construção *critical* no interior do fluxo constante do algoritmo destrói completamente as vantagens relativas da divisão de carga em CPUs multicore. O planejamento inteligente com alocação e agregação restritiva local (**private**) mitigado por adições globais ponderadas provê o equilíbrio definitivo entre integridade lógica computacional e a melhor escalabilidade matemática e temporal na computação paralela.

Anexo: Implementação C Completa

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include <time.h>

#define N 10000000

int main() {
    int count;
    double pi;
    double start, end;

    printf("Estimativa Estocastica de Pi (N = %d)\n\n", N);

    // =====
    // 1. Race Condition
    // =====
    count = 0;
    start = omp_get_wtime();
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        unsigned int seed = i;
        double x = (double)rand_r(&seed) / RAND_MAX;
```

```

        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0) {
            count++; // Condicao de corrida severa na variavel 'count'
        }
    }
    end = omp_get_wtime();
    pi = 4.0 * count / N;
    printf("1. Com Condicao de Corrida (#pragma omp parallel for):\n");
    printf("    Pi estimado: %f (Contagem: %d)\n", pi, count);
    printf("    Tempo: %f segundos\n\n", end - start);

    // =====
    // 2. Correcao Ineficiente com omp critical
    // =====
    count = 0;
    start = omp_get_wtime();
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        unsigned int seed = i;
        double x = (double)rand_r(&seed) / RAND_MAX;
        double y = (double)rand_r(&seed) / RAND_MAX;
        if (x * x + y * y <= 1.0) {
            #pragma omp critical
            {
                count++; // Mutex garante integridade, mas introduz um
pesado gargalo
            }
        }
    }
    end = omp_get_wtime();
    pi = 4.0 * count / N;
    printf("2. Correcao com #pragma omp critical (Sem otimizacao
local):\n");
    printf("    Pi estimado: %f (Contagem: %d)\n", pi, count);
    printf("    Tempo: %f segundos\n\n", end - start);

    // =====
    // 3. Reestruturacao com omp parallel, omp for e Clausulas
    // =====
    count = 0;
    int shared_var = 0;
    int first_priv_var = 100;
    int last_priv_var = -1;

    start = omp_get_wtime();
    /*
    * default(none) obriga que TODAS as variaveis externas sejam
    * explicitamente categorizadas como shared, private, etc.
    */
    #pragma omp parallel default(none) shared(count, shared_var,
last_priv_var) firstprivate(first_priv_var)
    {
        int local_count = 0; // Contagem local a cada thread (eficiencia
global)

```

```
double x, y;
unsigned int seed = omp_get_thread_num(); // Semente independente

#pragma omp for lastprivate(last_priv_var) private(x, y)
for (int i = 0; i < N; i++) {
    x = (double)rand_r(&seed) / RAND_MAX;
    y = (double)rand_r(&seed) / RAND_MAX;
    if (x * x + y * y <= 1.0) {
        local_count++;
    }
    if (i == N - 1) {
        // last_priv_var guarda na thread que fara a ultima
iteracao
        last_priv_var = omp_get_thread_num();
    }
}

// Agora o critical ocorre somente no repasse do somatorio
individual
#pragma omp critical
{
    count += local_count;
    shared_var += first_priv_var; // Acesso simultaneo em shared
}

end = omp_get_wtime();
pi = 4.0 * count / N;
printf("3. Reestruturado com Clausulas de Escopo:\n");
printf("  Pi estimado: %f (Contagem: %d)\n", pi, count);
printf("  Tempo: %f segundos\n", end - start);
printf("  Testando clausulas:\n");
printf("  - shared_var (mutada): %d (esperado: 100 * num_threads)\n",
shared_var);
printf("  - first_priv_var (externo): %d (esperado: 100)\n",
first_priv_var);
printf("  - last_priv_var (ID da thread da ultima iteracao): %d\n\n",
last_priv_var);

return 0;
}
```